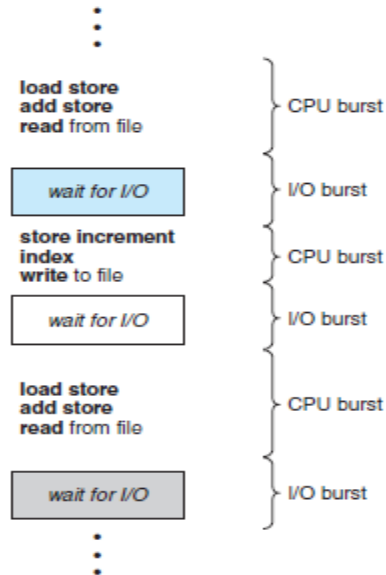


CPU SCHEDULING ALGORITHM:

CPU-I/O Burst Cycle:

- ❖ The process execution consists of a **cycle** of CPU execution and I/O wait. Processes alternate between these two states.
- ❖ Process execution begins with a **CPU burst**. That is followed by an **I/O burst**, which is followed by another CPU burst, then another I/O burst, and so on.
- ❖ Eventually, the final CPU burst ends with a system request to terminate execution.



Preemptive Scheduling and Non Preemptive scheduling:

- ❖ **Preemptive scheduling** allows a process to be interrupted in the midst of its execution, taking the CPU away and allocating it to another process.
- ❖ **Nonpreemptive scheduling** ensures that a process relinquishes control of the CPU only when it finishes with its current CPU burst.
- ❖ CPU-scheduling decisions may take place under the following four circumstances:
 - When a process switches from the running state to the waiting state
 - When a process switches from the running state to the ready state
 - When a process switches from the waiting state to the ready state
 - When a process terminates
- ❖ When scheduling takes place only under circumstances 1 and 4, we say that the scheduling scheme is **nonpreemptive** or **cooperative**. Otherwise, it is **preemptive**.

Scheduling Criteria:

The criteria include the following:

- **CPU utilization.** The CPU Should be kept as busy as possible for effective CPU utilization.
- **Throughput:** The total number of process completed per unit time is called as throughput.
- **Turnaround time:** The interval from the time of submission of a process to the time of completion is the turnaround time.

$$\text{Turnaround time} = \text{Waiting time} + \text{Burst time}$$

- **Waiting time:** Waiting time is the sum of the periods spent waiting in the ready queue.

$$\text{Waiting} = \text{time of completion} - \text{burst time}$$

- **Response time:** The time from the submission of a request until the first response is produced.

Scheduling Algorithms:

- ❖ CPU scheduling deals with the problem of deciding which of the processes in the ready queue is to be allocated the CPU. There are many different CPU-scheduling algorithms.
 - First-Come, First-Served Scheduling

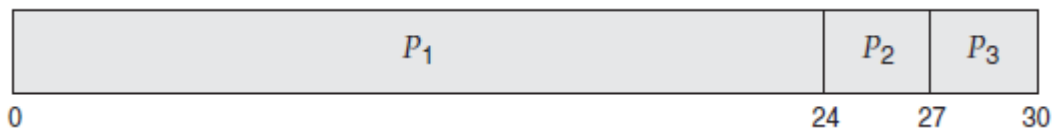
- Shortest-Job-First Scheduling
- Priority Scheduling
- Round-Robin Scheduling
- Multilevel Queue Scheduling
- Multilevel Feedback Queue Scheduling

FIRST COME FIRST SERVE SCHEDULING:

- ❖ With this scheme, the process that requests the CPU first is allocated the CPU first. The implementation of the FCFS policy is easily managed with a FIFO queue.
- ❖ When a process enters the ready queue, its PCB is linked onto the tail of the queue. When the CPU is free, it is allocated to the process at the head of the queue.
- ❖ The average waiting time under the FCFS policy is often quite long.
- ❖ **EXAMPLE:** Consider the following set of processes that arrive at time 0, with the length of the CPU burst given in milliseconds:

<u>Process</u>	<u>Burst Time</u>
P_1	24
P_2	3
P_3	3

- ❖ If the processes arrive in the order P_1, P_2, P_3 , and are served in FCFS order, we get the result shown in the following **Gantt chart**.



- ❖ The waiting time is 0 milliseconds for process P_1 , 24 milliseconds for process P_2 , and 27 milliseconds for process P_3 .
- ❖ Thus, the average waiting time is $(0 + 24 + 27)/3 = 17$ milliseconds.
- ❖ If the processes arrive in the order P_2, P_3, P_1 , however, the results will be as shown in the following Gantt chart:



- ❖ The average waiting time is now $(6 + 0 + 3)/3 = 3$ milliseconds. Thus, the average waiting time under an FCFS policy is generally not minimal.
- ❖ Assume that we have one CPU-bound process and many I/O-bound processes. The CPU-bound process will get and hold the CPU.
- ❖ During this time, all the other processes will finish their I/O and will move into the ready queue, waiting for the CPU.
- ❖ While the processes wait in the ready queue, the I/O devices are idle.
- ❖ Now the CPU-bound process finishes its CPU burst and moves to an I/O device. All the I/O-bound processes, which have short CPU bursts, execute quickly and move back to the I/O queues. Now the CPU sits idle. This is called as CONVOY EFFECT which results in lower CPU and device utilization.
- ❖ The FCFS scheduling algorithm is non preemptive. Once the CPU has been allocated to a process, that process keeps the CPU until it releases the CPU, either by terminating or by requesting I/O.
- ❖ **ADVANTAGES:**
 - Better for long processes
 - Simple method (i.e., minimum overhead on processor)
 - No starvation
 -
- ❖ **DISADVANTAGES:**

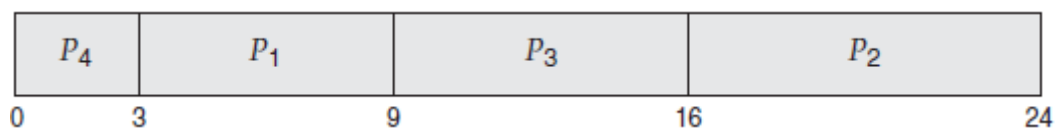
- Waiting time can be large if short requests wait behind the long ones.
- It is not suitable for time sharing systems where it is important that each user should get the CPU for an equal amount of time interval.
- A proper mix of jobs is needed to achieve good results from FCFS scheduling.

SHORTEST-JOB-FIRST SCHEDULING:

- ❖ With this algorithm the process that comes with the shortest job will be allocated the CPU first.
- ❖ This algorithm includes with each process the length of the process's next CPU burst. When the CPU is available, it is assigned to the process that has the smallest next CPU burst.
- ❖ If the next CPU bursts of two processes are the same, FCFS scheduling is used to break the tie.
- ❖ This is also called as shortest next CPU burst algorithm.
- ❖ **EXAMPLE:** consider the following set of processes, with the length of the CPU burst given in milliseconds:

Process	Burst Time
P_1	6
P_2	8
P_3	7
P_4	3

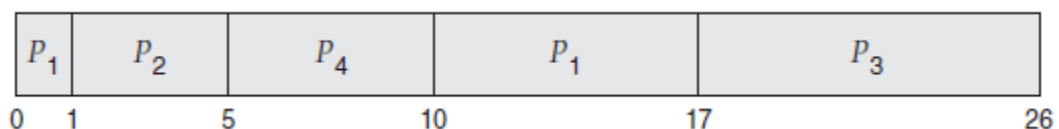
- ❖ Using SJF scheduling, we would schedule these processes according to the following Gantt chart:



- ❖ The waiting time is 3 milliseconds for process P_1 , 16 milliseconds for process P_2 , 9 milliseconds for process P_3 , and 0 milliseconds for process P_4 .
- ❖ Thus, the average waiting time is $(3 + 16 + 9 + 0)/4 = 7$ milliseconds.
- ❖ The SJF algorithm can be either preemptive or nonpreemptive.
- ❖ When a new process arrives at the ready queue while a previous process is still executing and the next CPU burst of the newly arrived process is shorter than what is left of the currently executing process, then a preemptive or non preemptive approach can be chosen.
- ❖ A preemptive SJF algorithm will preempt the currently executing process.
- ❖ A nonpreemptive SJF algorithm will allow the currently running process to finish its CPU burst. Preemptive SJF scheduling is sometimes called **shortest-remaining-time-first** scheduling.
- ❖ **EXAMPLE:** consider the following four processes, with the length of the CPU burst given in milliseconds:

Process	Arrival Time	Burst Time
P_1	0	8
P_2	1	4
P_3	2	9
P_4	3	5

- ❖ The preemptive SJF schedule is as depicted in the following Gantt chart:



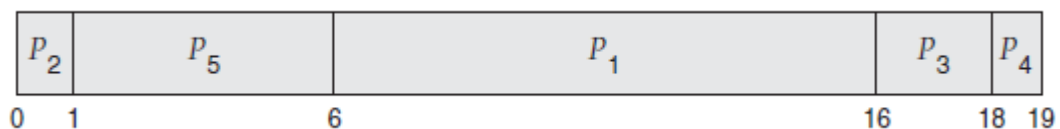
- ❖ Process P1 is started at time 0, since it is the only process in queue. Process P2 arrives at time 1.
- ❖ The remaining time for process P1 (7 milliseconds) is larger than the time required by process P2 (4 milliseconds), so process P1 is preempted, and process P2 is scheduled.
- ❖ The average waiting time is $[(10 - 1) + (1 - 1) + (17 - 2) + (5 - 3)]/4 = 26/4 = 6.5$ milliseconds.
- ❖ Nonpreemptive SJF scheduling would result in an average waiting time of 7.75 milliseconds
- ❖ **ADVANTAGES:**
 - The SJF scheduling algorithm has minimum average waiting time for a given set of processes.
 - Gives superior turnaround time performance to shortest process next because a short job is given immediate preference to a running longer job.
 - Throughput is high.
- ❖ **DISADVANTAGES:**
 - The difficulty with the SJF algorithm is knowing the length of the next CPU request
 - Starvation may be possible for the longer processes.

PRIORITY SCHEDULING:

- ❖ A priority is associated with each process, and the CPU is allocated to the process with the highest priority.
- ❖ Equal-priority processes are scheduled in FCFS order.
- ❖ An SJF algorithm is simply a priority algorithm where the priority (p) is the inverse of the (predicted) next CPU burst.
- ❖ As an example, consider the following set of processes, assumed to have arrived at time 0 in the order P1, P2, ..., P5, with the length of the CPU burst given in milliseconds:

<u>Process</u>	<u>Burst Time</u>	<u>Priority</u>
P_1	10	3
P_2	1	1
P_3	2	4
P_4	1	5
P_5	5	2

- ❖ Using priority scheduling, we would schedule these processes according to the following Gantt chart:



- ❖ Thus, the average waiting time is $(6 + 0 + 16 + 18 + 1)/5 = 8.2$ milliseconds
- ❖ Priorities can be defined either internally or externally
- ❖ Priority scheduling can be either preemptive or nonpreemptive. When a process arrives at the ready queue, its priority is compared with the priority of the currently running process.
- ❖ A preemptive priority scheduling algorithm will preempt the CPU if the priority of the newly arrived process is higher than the priority of the currently running process.
- ❖ A nonpreemptive priority scheduling algorithm will simply put the new process at the head of the ready queue.
- ❖ A process that is ready to run but waiting for the CPU can be considered blocked. A priority scheduling algorithm can leave some low priority processes waiting indefinitely. Hence the higher-priority processes can prevent a low-priority process from ever getting the CPU. This problem with priority scheduling algorithms is **indefinite blocking**, or **starvation**.
- ❖ A solution to the problem of indefinite blockage of low-priority processes is **aging**.
- ❖ Aging involves gradually increasing the priority of processes that wait in the system for a long time.

❖ **ADVANTAGES:**

1. Simplicity.
2. Reasonable support for priority.
3. Suitable for applications with varying time and resource requirements.

❖ **DISADVANTAGES**

1. Indefinite blocking or starvation.
2. A priority scheduling can leave some low priority waiting processes indefinitely for CPU.
3. If the system eventually crashes then all unfinished low priority processes gets lost.

ROUND ROBIN SCHEDULING:

- ❖ The **round-robin (RR)** scheduling algorithm is designed especially for timesharing systems.
- ❖ A small unit of time, called a **time quantum** or **time slice**, is defined. A time quantum is generally from 10 to 100 milliseconds in length.
- ❖ The CPU scheduler picks the first process from the ready queue, sets a timer to interrupt after 1 time quantum, and dispatches the process.
- ❖ The process may have a CPU burst of less than 1 time quantum. In this case, the process itself will release the CPU voluntarily. The scheduler will then proceed to the next process in the ready queue.

- ❖ **EXAMPLE:** Consider the following set of processes that arrive at time 0, with the length of the CPU burst given in milliseconds:

<u>Process</u>	<u>Burst Time</u>
P_1	24
P_2	3
P_3	3

P_1	P_2	P_3	P_1	P_1	P_1	P_1	P_1	
0	4	7	10	14	18	22	26	30

- ❖ If we use a time quantum of 4 milliseconds, then process P_1 gets the first 4 milliseconds. Since it requires another 20 milliseconds, it is preempted after the first time quantum, and the CPU is given to the next process in the queue, process P_2 .
- ❖ Process P_2 does not need 4 milliseconds, so it quits before its time quantum expires.
- ❖ The CPU is then given to the next process, process P_3 . Once each process has received 1 time quantum, the CPU is returned to process P_1 for an additional time quantum.
- ❖ P_1 waits for 6 milliseconds (10 - 4), P_2 waits for 4 milliseconds, and P_3 waits for 7 milliseconds.
- ❖ Thus, the average waiting time is $17/3 = 5.66$ milliseconds.
- ❖ The performance of the RR algorithm depends heavily on the size of the time quantum.
- ❖ If the time quantum is extremely large, the RR policy is the same as the FCFS policy.
- ❖ If the time quantum is extremely small the RR approach can result in a large number of context switches.

❖ **ADVANTAGES:**

- Does not suffer by starvation.

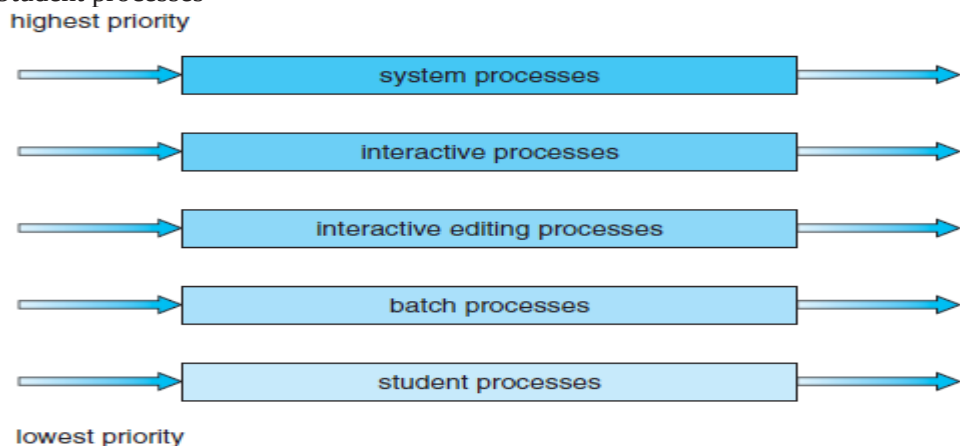
❖ **DISADVANTAGE:**

- The average waiting time under the RR policy is often long.
- There is Low throughput.
- There are Context Switches.
-

MULTI LEVEL QUEUE SCHEDULING:

- ❖ The processes are divided into foreground process and background process.

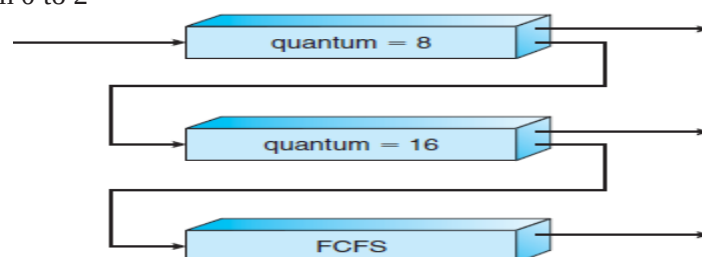
- ❖ These two types of processes have different response-time requirements and have different scheduling needs.
- ❖ The foreground processes may have priority over background processes.
- ❖ A **multilevel queue** scheduling algorithm partitions the ready queue into several separate queues.
- ❖ The processes are permanently assigned to one queue, generally based on some property of the process, such as memory size, process priority, or process type.
- ❖ Example of a multilevel queue scheduling algorithm with five queues, in order of priority:
 1. System processes
 2. Interactive processes
 3. Interactive editing processes
 4. Batch processes
 5. Student processes



- ❖ Each queue has its own scheduling algorithm. For example, separate queues might be used for foreground and background processes.
- ❖ The foreground queue might be scheduled by an RR algorithm, while the background queue is scheduled by an FCFS algorithm.
- ❖ There must be scheduling among the queues, which is commonly implemented as fixed-priority preemptive scheduling.
- ❖ No process in the batch queue, for example, could run unless the queues for system processes, interactive processes, and interactive editing processes were all empty.
- ❖ If an interactive editing process entered the ready queue while a batch process was running, the batch process would be preempted.

MULTILEVEL FEEDBACK QUEUE SCHEDULING:

- ❖ The **multilevel feedback queue** scheduling algorithm, allows a process to move between queues.
- ❖ The idea is to separate processes according to the characteristics of their CPU bursts.
- ❖ If a process uses too much CPU time, it will be moved to a lower-priority queue.
- ❖ A process that waits too long in a lower-priority queue may be moved to a higher-priority queue. This form of aging prevents starvation.
- ❖ EXAMPLE: For example, consider a multilevel feedback queue scheduler with three queues, numbered from 0 to 2



- ❖ The scheduler first executes all processes in queue 0. Only when queue 0 is empty will it execute processes in queue 1. Similarly, processes in queue 2 will be executed only if queues 0 and 1 are empty.
- ❖ A process that arrives for queue 1 will preempt a process in queue 2. A process in queue 1 will in turn be preempted by a process arriving for queue 0.
- ❖ A process entering the ready queue is put in queue 0. A process in queue 0 is given a time quantum of 8 milliseconds.
- ❖ If it does not finish within this time, it is moved to the tail of queue 1. If queue 0 is empty, the process at the head of queue 1 is given a quantum of 16 milliseconds.
- ❖ If it does not complete, it is preempted and is put into queue 2. Processes in queue 2 are run on an FCFS basis but are run only when queues 0 and 1 are empty. a multilevel feedback queue